

Analysis Modeling

1

1

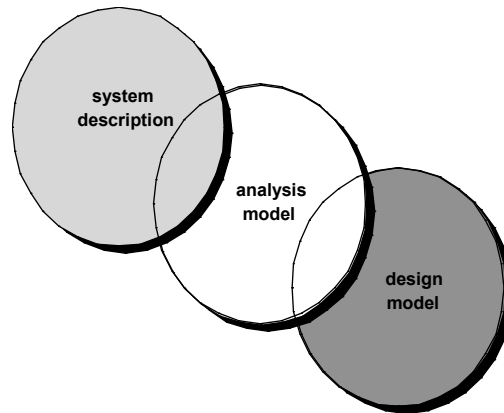
Requirements Analysis

- Requirements analysis
 - specifies software's operational characteristics
 - indicates software's interface with other system elements
 - establishes constraints that software must meet
- Requirements analysis allows the software engineer (called an *analyst* or *modeler* in this role) to:
 - elaborate on basic requirements established during earlier requirement engineering tasks
 - build models that depict user scenarios, functional activities, problem classes and their relationships, system and class behavior, and the flow of data as it is transformed.

2

2

A Bridge



3

3

Analysis Modeling Approaches

- **Structure analysis** considers data and the process that transform the data as separate entities.
 - Data objects are modeled in a way that defines their attributes and relationships.
 - Process that manipulate data objects are modeled in a manner that shows how they transform data as data objects flow through the system.
- **Object-oriented analysis** focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements.

4

4

Data Modeling

- examines data objects independently of processing
- focuses attention on the data domain
- creates a model at the customer's level of abstraction
- indicates how data objects relate to one another

5

5

Flow-Oriented Modeling

- Represents how data objects are transformed as they move through the system
- A data flow diagram (DFD) is the diagrammatic form that is used
- Considered by many to be an 'old school' approach, flow-oriented modeling continues to provide a view of the system that is unique—it should be used to supplement other analysis model elements

6

6

The Flow Model

Every computer-based system is an information transform



7

7

Data Flow Diagram Elements

- Process
 - An activity or function performed for a specific business reason
 - Manual or computerized
- Data flow
 - A single piece of data or a logical collection of data
 - Always starts or ends at a process

8

8

DFD Elements (cont'd)

- Data Store
 - A collection of data that is stored in some way
 - Data flowing out is retrieved from the data store
 - Data flowing in updates or is added to the data store
- External entity
 - A person, organization, or system that is *external* to the system but interacts with it.

9

9

Naming and Drawing DFD Elements

	Data Flow Diagram Element	Typical Computer-Aided Software Engineering Fields	Gane and Sarson Symbol	DeMarco and Yourdan Symbol
Process	Every <i>process</i> has A number A name (verb phrase) A description One or more output data flows Usually one or more input data flows	Label (name) Type (process) Description (what is it) Process number Process description (Structured English) Notes		
Data flow	Every <i>data flow</i> has A name (a noun) A description One or more connections to a process	Label (name) Type (flow) Description Alias (another name) Composition (description of data elements) Notes		
Data store	Every <i>data store</i> has A number A name (a noun) A description One or more input data flows Usually one or more output data flows	Label (name) Type (store) Description Alias (another name) Composition (description of data elements) Notes		
External entity	Every <i>external entity</i> has A name (a noun) A description	Label (name) Type (entity) Description Alias (another name) Entity description Notes		

10

10

Depicting Business Processes with DFDs

- Business processes are too complex to be shown on a single DFD
- *Decomposition* is the process of representing the system in a hierarchy of DFD diagrams
 - Child diagrams show a portion of the parent diagram in greater detail

11

11

Relationship Among DFD levels

Context diagram



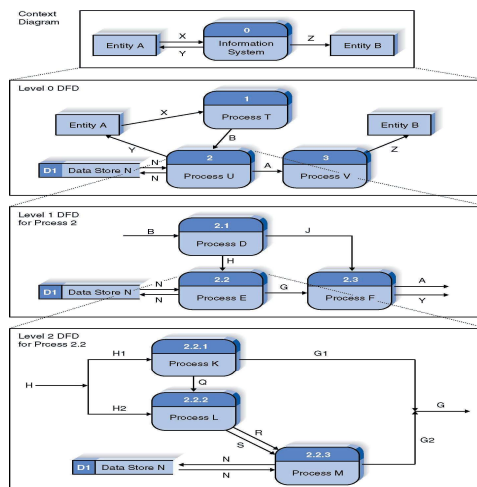
Level 0 diagram



Level 1 diagram



Level 2 diagram



12

12

Integrating Scenario Descriptions

- DFDs start with the use cases and requirements definition
- Generally, the DFDs integrate the use cases
- Names of use cases become processes
- Inputs and outputs become data flows
- “Small” data inputs and outputs are combined into a single flow

13

13

Validating the DFD

- Syntax errors – diagram follows the rules
 - Assure correct DFD structure

For each DFD:

Check each **process** for:

A unique name: action verb phrase; number; description
At least one input data flow
At least one output data flow
Output data flow names usually different than input data flow names
Between 3 and 7 processes per DFD

14

14

Validating the DFD (cont'd)

For each DFD:

Check each **data flow** for:

- A unique name: noun; description
- Connects to at least one process
- Shown in only one direction (no two-headed arrows)
- A minimum number of crossed lines

Check each **data store** for:

- A unique name: noun; description
- At least one input data flow
- At least one output data flow

Check each **external entity** for:

- A unique name: noun; description
- At least one input or output data flow

15

15

Validating the DFD (cont'd)

Across DFDs:

Context Diagram:

- Every set of DFDs must have one Context Diagram

Viewpoint:

- There is a consistent viewpoint for the entire set of DFDs

Decomposition:

- Every process is wholly and complete described by the processes on its children DFDs

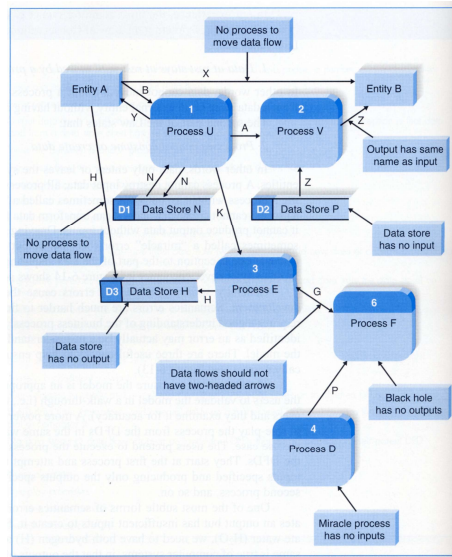
Balance:

- Every data flow, data store, and external entity on a higher level DFD is shown on the lower level DFD that decomposes it
- No data stores or data flows appear on lower-level DFDs that do not appear on their parent DFD

16

16

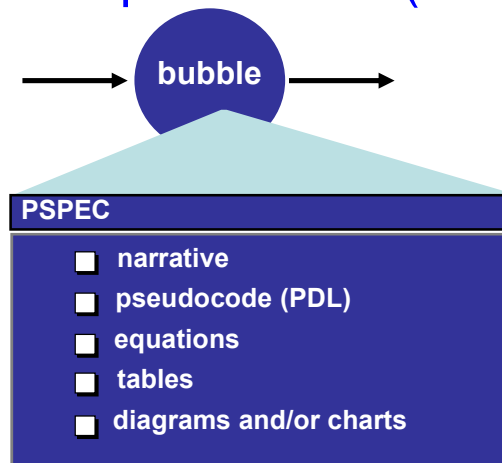
Some Common Errors



17

17

Process Specification (PSPEC)



18

18

Object-Oriented Concepts

- Must be understood to apply class-based elements of the analysis model
- Key concepts:
 - Classes and objects
 - Attributes and operations
 - Encapsulation and instantiation
 - Inheritance

19

19

Classes-based Modeling

20

20

Classes

- object-oriented thinking begins with the definition of a class, often defined as:
 - template
 - “blueprint” ... describing a collection of similar items class can be identified

21

21

Analysis Classes

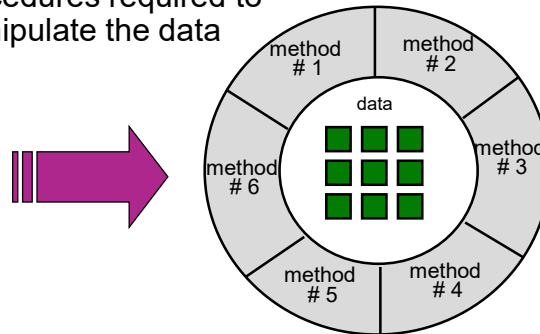
- *External entities* (e.g., other systems, devices, people) that produce or consume information to be used by a computer-based system.
- *Things* (e.g., reports, displays, letters, signals) that are part of the information domain for the problem.
- *Occurrences or events* (e.g., a property transfer or the completion of a series of robot movements) that occur within the context of system operation.
- *Roles* (e.g., manager, engineer, salesperson) played by people who interact with the system.
- *Organizational units* (e.g., division, group, team) that are relevant to an application.
- *Places* (e.g., manufacturing floor or loading dock) that establish the context of the problem and the overall function of the system.
- *Structures* (e.g., sensors, four-wheeled vehicles, or computers) that define a class of objects or related classes of objects.

22

22

Encapsulation/Hiding

The object encapsulates both data and the logical procedures required to manipulate the data



23

23

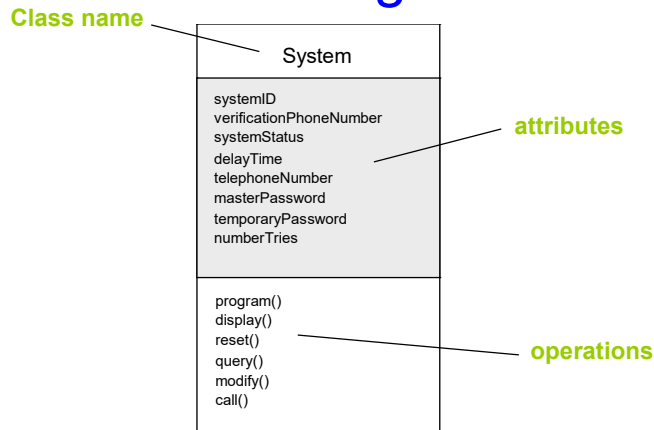
Methods (a.k.a. Operations, Services)

An executable procedure that is encapsulated in a class and is designed to operate on one or more data attributes that are defined as part of the class. A method is invoked via message passing.

24

24

Class Diagram



25

25

Class Types

- Entity classes, also called model or business classes, are extracted directly from the statement of the problem (e.g., FloorPlan and Sensor).
- Boundary classes are used to create the interface (e.g., interactive screen or printed reports) that the user sees and interacts with as the software is used.
- Controller classes manage a “unit of work” from start to finish. That is, controller classes can be designed to manage

26

26

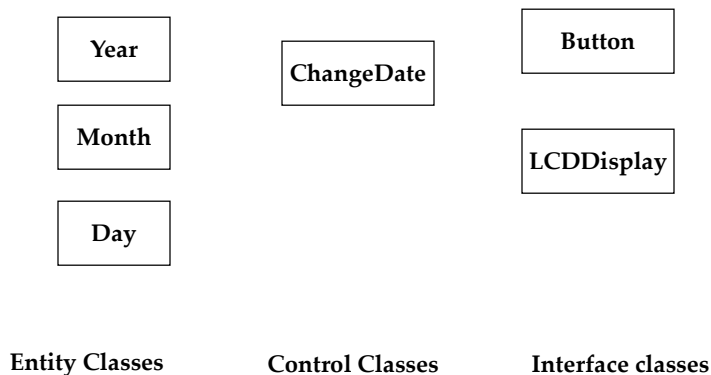
Class Types (cont'd)

- the creation or update of entity objects;
- the instantiation of boundary objects as they obtain information from entity objects;
- complex communication between sets of objects;
- validation of data communicated between objects or between the user and the application.

27

27

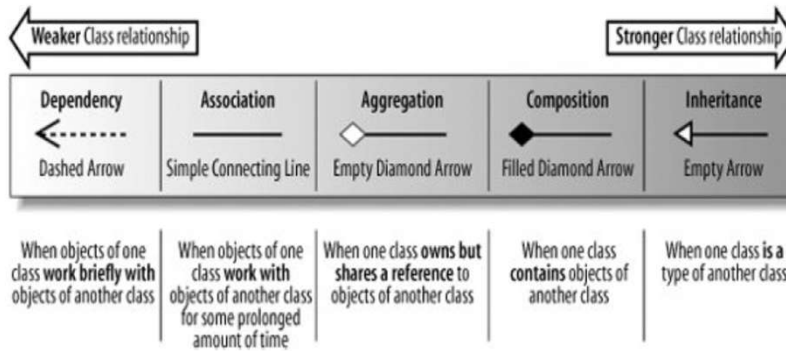
Example: 2BWatch Classes



28

28

Class relationships



29

29

Scenario-Based Modeling

30

30

Scenario-Based Modeling

“[Use-cases] are simply an aid to defining what exists outside the system (actors) and what should be performed by the system (use-cases).” Ivar Jacobson

31

31

Use-Cases

- a scenario that describes a “thread of usage” for a system
- actors represent roles people or devices play as the system functions
- users can play a number of different roles for a given scenario

32

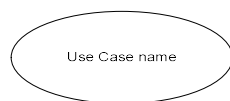
32

Use Case Diagram

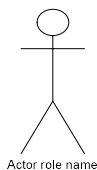
- Summarizes all use cases (for the part of the system being modeled) together in one picture
- Typically drawn early in the SDLC
- Shows the associations between actors and use cases

33

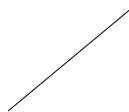
33



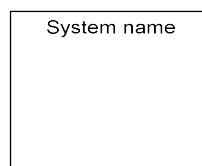
Use case: represent a major piece of system functionality



Actor: a person or system that derives benefit from and is external to the system



Communication line: link an actor with the use case(s) with which it interact



System boundary: includes the name of the system inside or on top
Represents the scope the system

34

34

Use case

The important thing to remember is that a use case from the user's perspective is a complete use of the system

Communication Lines

- A communication line connects an actor and a use case to show the actor participating in the use case
- The purpose of a communication line is to show that an actor is simply involved in a use case, not to imply an information exchange in any particular direction or that the actor starts the use case.

35

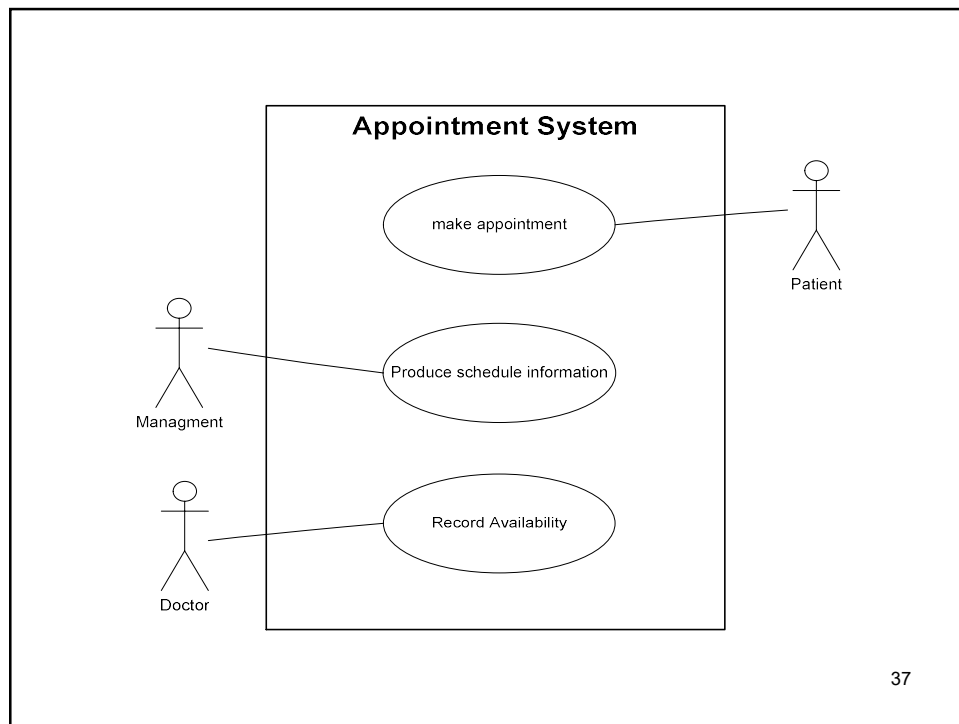
35

System boundary

To show your system's boundary on a use case diagram, draw a box around all of the use cases but keep the actors outside of the box.

36

36



37

Use Case Relationship

- There is some similarity between steps in different use cases.
- Get rid of the repetition between use case descriptions.
- Show reusable and optional use case behavior between use cases.

38

38

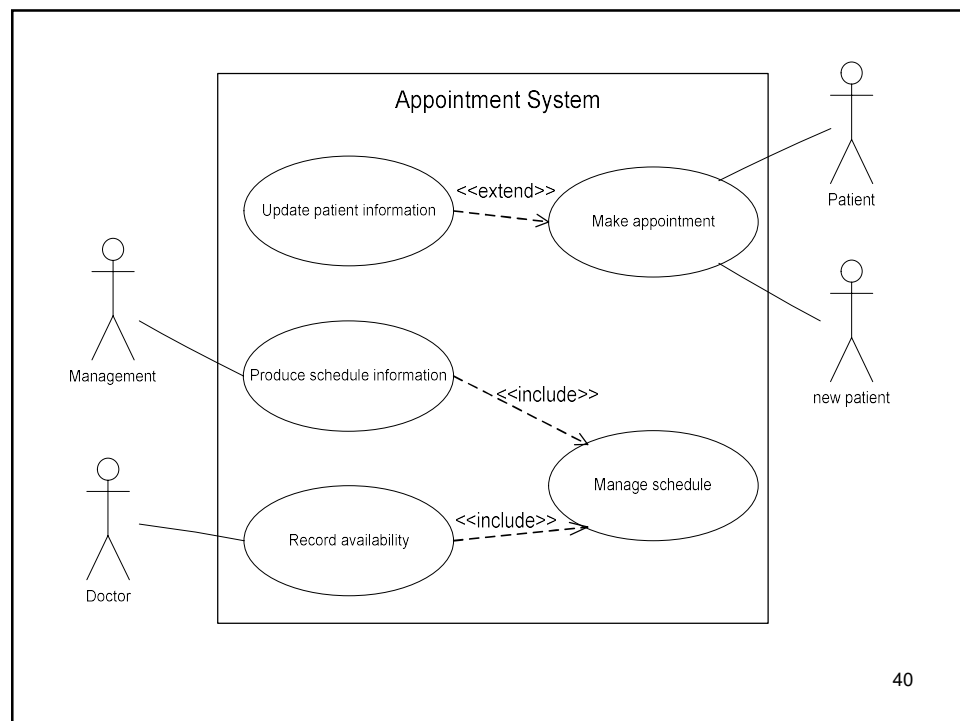
Use Case Relationship (cont'd)

The <<include>> Relationship

The <<include>> relationship declares that the use case at the tail of the dotted arrow completely reuses all of the steps from the use case being included.

39

39



40

40

Use Case Relationship (cont'd)

The <<extend>> Relationship

The <<extend>> relationship shows that a use case might completely reuse another use case's behavior, similar to the <<include>> relationship, but that this reuse was optional and dependent either on a runtime or system implementation decision.

41

41

Behavioral Modeling

42

42

Behavioral Modeling

- make a list of the different states of a system (How does the system behave?)
- indicate how the system makes a transition from one state to another (How does the system change state?)
 - indicate event
 - indicate action
- draw a state diagram or a sequence diagram

43

43

Sequence Diagram

- From the flow of events in the use case or scenario proceed to the sequence diagram
- A sequence diagram is a graphical description of objects participating in a use case or scenario using a DAG (direct acyclic graph) notation
- Relation to object identification:
 - Objects/classes have already been identified during object modeling
 - Objects are identified as a result of dynamic modeling

44

44

Sequence Diagram (cont'd)

- Heuristic:
 - An event always has a sender and a receiver.
 - The representation of the event is sometimes called a message
 - Find them for each event => These are the objects participating in the use case

45

45

Heuristics for Sequence Diagrams

- Layout:
 - 1st column: Should correspond to the actor who initiated the use case
 - 2nd column: Should be a boundary object
 - 3rd column: Should be the control object that manages the rest of the use case
- Creation:
 - Control objects are created at the initiation of a use case
 - Boundary objects are created by control objects

46

46

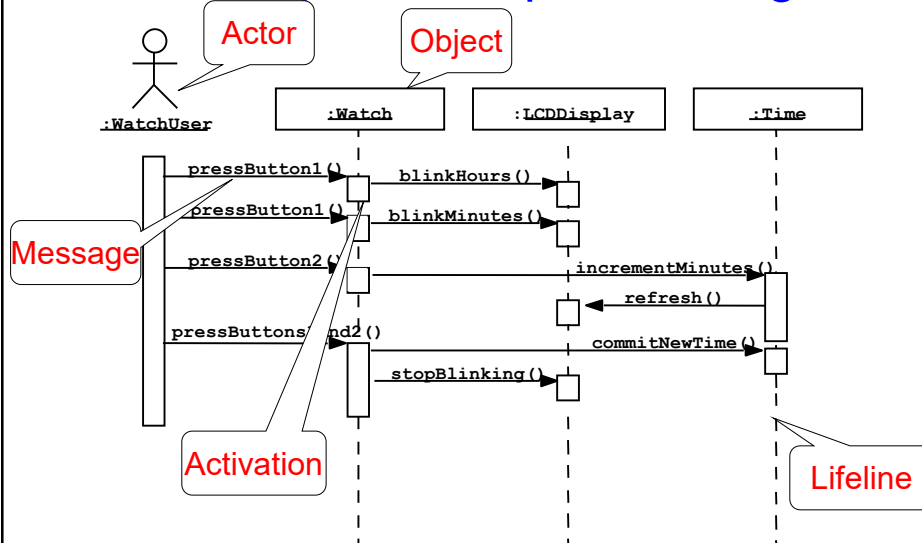
Heuristics for Sequence Diagrams (cont'd)

- Access:
 - Entity objects are accessed by control and boundary objects,
 - Entity objects should never call boundary or control objects

47

47

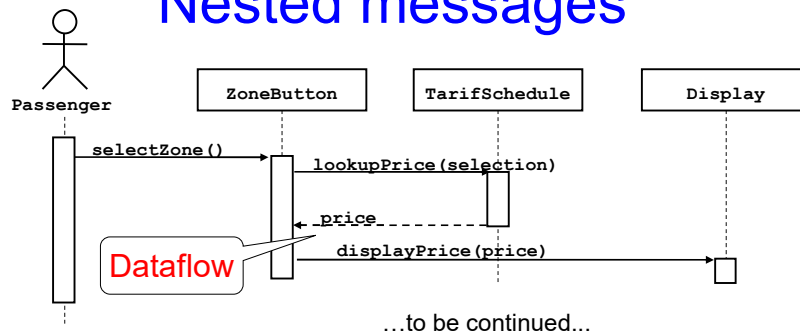
UML first pass: Sequence diagram



Sequence diagrams represent the behavior as interactions³

48

Nested messages

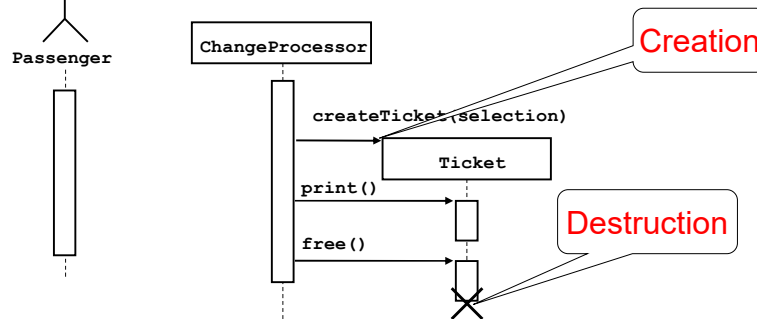


- The source of an arrow indicates the activation which sent the message
- An activation is as long as all nested activations
- Horizontal dashed arrows indicate data flow
- Vertical dashed lines indicate lifelines

49

49

Creation and destruction

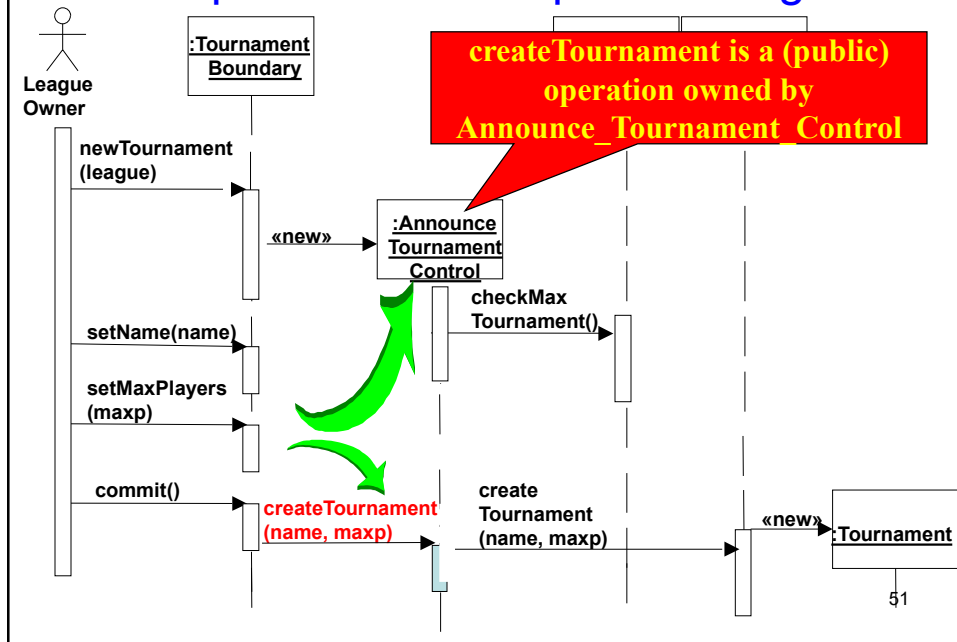


- Creation is denoted by a message arrow pointing to the object.
- Destruction is denoted by an X mark at the end of the destruction activation.
- In garbage collection environments, destruction can be used to denote the end of the useful life of an object.

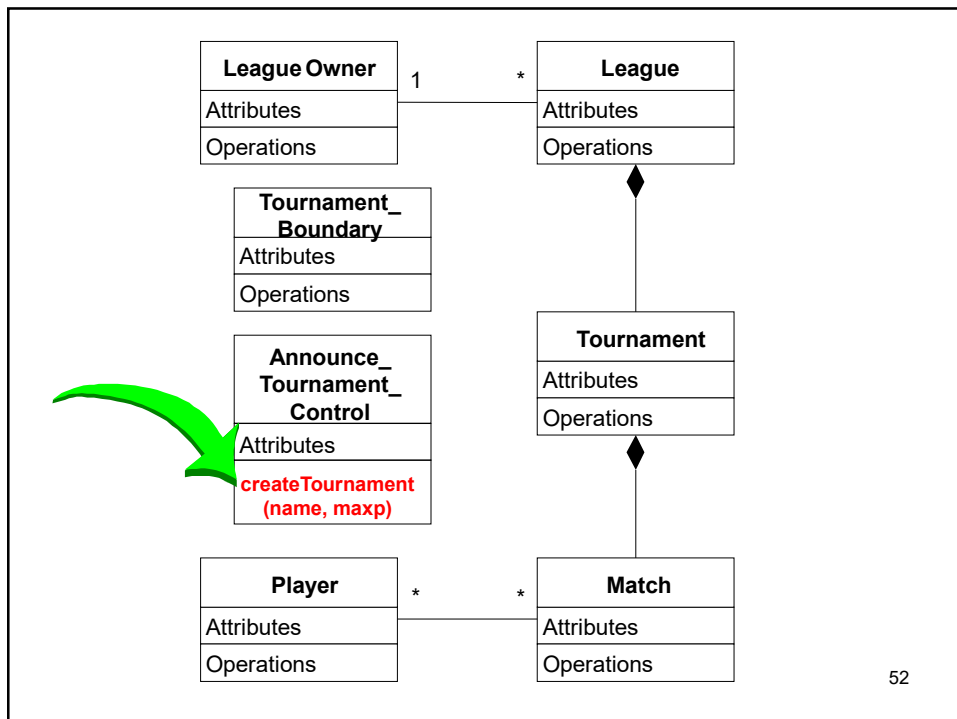
50

50

Example from the Sequence Diagram



51

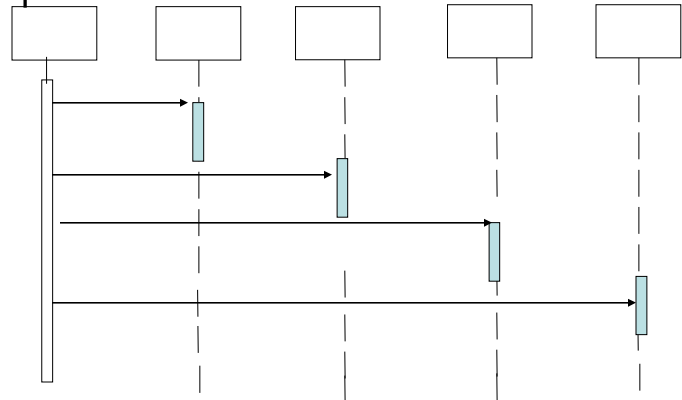


52

52

Fork Diagram

- Much of the dynamic behavior is placed in a single object, usually the control object. It knows all the other objects and often uses them for direct questions and commands.

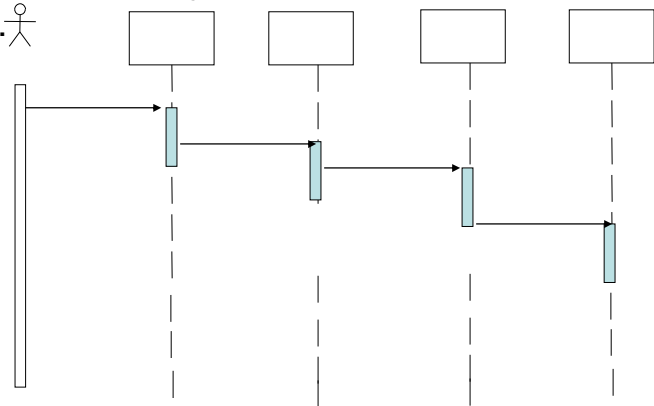


53

53

Stair Diagram

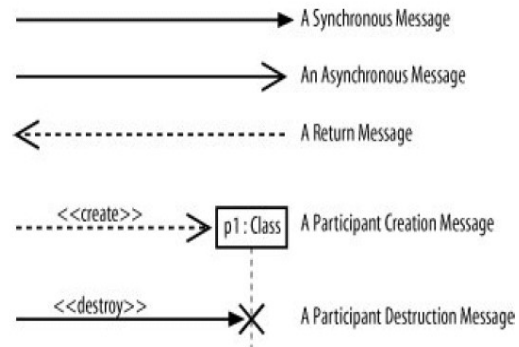
- The dynamic behavior is distributed. Each object delegates some responsibility to other objects. Each object knows only a few of the other objects and knows which objects can help with a specific behavior.



54

54

Sequence Diagram



55

55

Q&A

56

56